

09-testing

09. Testing (Elektif)

Level: □ Intermediate

Jam: 6 (3 sesi)

Prasyarat: Node.js & Express, TypeScript dasar

Output: Test suite unit + integration berjalan otomatis via CI

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Memahami **test mindset**: kenapa testing penting, apa itu test coverage
- Nulis **unit test** pakai Vitest (describe/it/expect, matchers, mocking)
- Nulis **integration test** buat Express API pakai supertest
- Ngerjain **CI/CD pipeline** dengan GitHub Actions yang auto-run test
- Baca & debug test failure dari log CI

Materi

Sesi	Topik	File
1	Unit Test — Vitest (matchers, mocking, async)	01-unit-test.md
2	Integration Test — Supertest, DB, auth	02-integration-test.md
3	CI/CD — GitHub Actions, coverage, matrix	03-ci-cd.md

Output Akhir Modul

Test Suite Otomatis CI — Repo dengan test unit + integration yang jalan otomatis tiap push via GitHub Actions, lengkap dengan coverage report.

AI Prompt Exercises

Sepanjang modul, latihan pake AI:

- "Explain this test case: what does it test and what could go wrong?"
- "Generate edge cases for this function I need to test"
- "Why did this test fail? Here's the error log: ..."
- "Refactor this test to use `test.each` for parameterized testing"
- "Generate a GitHub Actions workflow that runs tests on push and PR"

1. Unit Test dengan Vitest

Setup Vitest

```
npm init -y
npm install -D vitest typescript @types/node
npx tsc --init
```

package.json — tambah script test:

```
{
  "scripts": {
    "test": "vitest run",
    "test:watch": "vitest"
  }
}
```

Bikin file calculator.ts:

```
export const add = (a: number, b: number): number => a + b;

export const divide = (a: number, b: number): number => {
  if (b === 0) throw new Error("Cannot divide by zero");
  return a / b;
};

export const getMax = (arr: number[]): number => {
  return Math.max(...arr);
};
```

describe / it / expect

```
// calculator.test.ts
import { describe, it, expect } from "vitest";
import { add, divide, getMax } from "../calculator";
```

```
describe("add", () => {
  it("adds two positive numbers", () => {
    expect(add(2, 3)).toBe(5);
  });

  it("adds negative numbers", () => {
    expect(add(-1, -2)).toBe(-3);
  });

  it("adds zero", () => {
    expect(add(5, 0)).toBe(5);
  });
});
```

Matchers

Matcher	Fungsi
toBe(val)	=== strict equality (primitives)
toEqual(obj)	Deep equality (object/array)
toContain(item)	Array/string contains
toThrow(err?)	Function throws
toBeGreaterThan(n)	> n
toBeDefined()	Not undefined

```
describe("matchers demo", () => {
  it("toEqual for objects", () => {
    expect({ a: 1, b: 2 }).toEqual({ a: 1, b: 2 });
  });

  it("toContain in array", () => {
    expect([1, 2, 3]).toContain(2);
  });

  it("toThrow on error", () => {
    expect(() => divide(1, 0)).toThrow("Cannot divide by zero");
  });

  it("toBeGreaterThan", () => {
    expect(getMax([1, 5, 3])).toBeGreaterThan(4);
  });
});
```

Mocking

`vi.fn()` — bikin spy function:

```
import { vi, describe, it, expect } from "vitest";

describe("vi.fn", () => {
  it("tracks calls", () => {
    const mock = vi.fn((x: number) => x * 2);

    expect(mock(3)).toBe(6);
    expect(mock).toHaveBeenCalledTimes(3);
    expect(mock).toHaveBeenCalledTimes(1);
  });
});
```

`vi.mock()` — mock entire module:

```
// userService.ts
import { db } from "../db";

export const getUser = async (id: string) => {
  return db.query("SELECT * FROM users WHERE id = $1", [id]);
};

// userService.test.ts
import { vi, describe, it, expect } from "vitest";
import { getUser } from "../userService";

vi.mock("../db", () => ({
  db: {
    query: vi.fn().mockResolvedValue([{ id: "1", name: "Budi" }]),
  },
}));

describe("getUser", () => {
  it("returns user from mocked db", async () => {
    const user = await getUser("1");
    expect(user).toEqual([{ id: "1", name: "Budi" }]);
  });
});
```

Testing Pure Functions

Pure function = input sama -> output sama, no side effects. Paling gampang di-test.

```

// utils.ts
export const toRupiah = (amount: number): string => {
  return new Intl.NumberFormat("id-ID", {
    style: "currency",
    currency: "IDR",
  }).format(amount);
};

export const filterActive = <T extends { active: boolean }>(items: T[]): T[] =>
  return items.filter((item) => item.active);
};

// utils.test.ts
import { describe, it, expect } from "vitest";
import { toRupiah, filterActive } from "../utils";

describe("toRupiah", () => {
  it("formats number to IDR", () => {
    expect(toRupiah(15000)).toBe("Rp15.000,00");
  });

  it("handles zero", () => {
    expect(toRupiah(0)).toBe("Rp0,00");
  });
});

describe("filterActive", () => {
  it("keeps only active items", () => {
    const items = [
      { id: 1, active: true },
      { id: 2, active: false },
      { id: 3, active: true },
    ];
    expect(filterActive(items)).toHaveLength(2);
  });

  it("returns empty array if none active", () => {
    expect(filterActive([{ id: 1, active: false }])).toEqual([]);
  });
});

```

Async Tests

```

// api.ts
export const fetchUser = async (id: number): Promise<{ name: string }> => {
  const res = await fetch(`https://jsonplaceholder.typicode.com/users/${id}`);

```

```

    if (!res.ok) throw new Error("Network error");
    return res.json();
  };
}

// api.test.ts
import { describe, it, expect, vi } from "vitest";
import { fetchUser } from "../api";

describe("fetchUser", () => {
  it("returns user data on success", async () => {
    const mockData = { name: "Budi" };
    vi.spyOn(global, "fetch").mockResolvedValueOnce({
      ok: true,
      json: async () => mockData,
    } as Response);

    const user = await fetchUser(1);
    expect(user).toEqual(mockData);
  });

  it("throws on network error", async () => {
    vi.spyOn(global, "fetch").mockResolvedValueOnce({
      ok: false,
    } as Response);

    await expect(fetchUser(1)).rejects.toThrow("Network error");
  });
});

```

Latihan

1. **Calculator test lengkap** — Bikin function multiply, subtract, isEven. Test semua: positive, negative, zero, edge case (kalo isEven pake bilangan ganjil besar)
2. **Shopping cart test** — Bikin function calculateTotal(items: {price: number, qty: number}[]) → total price. Test: cart kosong, cart isi 1 item, cart isi banyak item, kalo qty 0
3. **String utility test** — Bikin function capitalizeWords(str: string): string. Test: string kosong, satu kata, beberapa kata, all caps input, mixed case input
4. **Async user search** — Bikin function searchUsers(keyword: string) yang pake fetch ke dummy API. Test: mock fetch sukses return array, mock fetch kosong, mock fetch error. Pake vi.spyOn + mockResolvedValueOnce

2. Integration Test dengan Supertest

Setup

Install dependensi:

```
npm install express  
npm install -D vitest supertest @types/express @types/supertest
```

Testing Express Routes

Bikin Express app minimal:

```
// app.ts  
import express from "express";  
  
const app = express();  
app.use(express.json());  
  
interface Todo {  
  id: number;  
  title: string;  
  done: boolean;  
}  
  
let todos: Todo[] = [];  
let nextId = 1;  
  
app.get("/api/todos", (_req, res) => {  
  res.json(todos);  
});  
  
app.post("/api/todos", (req, res) => {  
  const { title } = req.body;  
  if (!title || typeof title !== "string") {  
    return res.status(400).json({ error: "Title is required" });  
  }  
  const todo: Todo = { id: nextId++, title, done: false };  
  todos.push(todo);  
  res.status(201).json(todo);  
});  
  
app.put("/api/todos/:id", (req, res) => {  
  const id = Number(req.params.id);  
  const todo = todos.find((t) => t.id === id);  
  if (!todo) return res.status(404).json({ error: "Not found" });
```

```

    todo.done = req.body.done ?? todo.done;
    todo.title = req.body.title ?? todo.title;
    res.json(todo);
  });

  app.delete("/api/todos/:id", (req, res) => {
    const id = Number(req.params.id);
    const idx = todos.findIndex((t) => t.id === id);
    if (idx === -1) return res.status(404).json({ error: "Not found" });
    todos.splice(idx, 1);
    res.status(204).send();
  });

```

```
export { app };
```

Test pakai supertest:

```
// app.test.ts
```

```
import { describe, it, expect } from "vitest";
import supertest from "supertest";
import { app } from "../app";
```

```
const request = supertest(app);
```

```
describe("GET /api/todos", () => {
  it("returns empty array initially", async () => {
    const res = await request.get("/api/todos");
    expect(res.status).toBe(200);
    expect(res.body).toEqual([]);
  });
});
```

```
describe("POST /api/todos", () => {
  it("creates a new todo", async () => {
    const res = await request.post("/api/todos").send({ title: "Belajar testing" });
    expect(res.status).toBe(201);
    expect(res.body.title).toBe("Belajar testing");
    expect(res.body.done).toBe(false);
    expect(res.body.id).toBeDefined();
  });
});
```

```
it("rejects empty title", async () => {
  const res = await request.post("/api/todos").send({});
  expect(res.status).toBe(400);
  expect(res.body.error).toBe("Title is required");
});
});
```

PENTING: Antar-test bisa saling ganggu kalo pake shared state (array todos). Solusi: reset tiap test pake `beforeEach`. Karena kita pake module-level variabel, perlu export fungsi reset atau bikin ulang app tiap test.

Testing DB dengan In-Memory

Alternatif: pake database ringan (SQLite in-memory) biar test ga kena data produksi.

```
// db.ts
import Database from "better-sqlite3";

// Di production pake file, di test pake ':memory:'
export const createDb = (filename: string = ":memory:") => {
  const db = new Database(filename);
  db.exec(`
    CREATE TABLE IF NOT EXISTS todos (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      title TEXT NOT NULL,
      done INTEGER DEFAULT 0
    )
  `);
  return db;
};

// todoRepository.ts
import Database from "better-sqlite3";

export class TodoRepository {
  constructor(private db: Database.Database) {}

  findAll() {
    return this.db.prepare("SELECT * FROM todos").all();
  }

  create(title: string) {
    const stmt = this.db.prepare("INSERT INTO todos (title) VALUES (?)");
    const info = stmt.run(title);
    return this.db.prepare("SELECT * FROM todos WHERE id = ?").get(info.lastInsertId);
  }
}

// todoRepository.test.ts
import { describe, it, expect, beforeEach, afterEach } from "vitest";
import Database from "better-sqlite3";
import { TodoRepository } from "../todoRepository";
```

```

import { createDb } from "../db";

describe("TodoRepository", () => {
  let db: Database.Database;
  let repo: TodoRepository;

  beforeEach(() => {
    db = createDb(); // in-memory
    repo = new TodoRepository(db);
  });

  afterEach(() => {
    db.close();
  });

  it("findAll returns empty initially", () => {
    expect(repo.findAll()).toEqual([]);
  });

  it("create inserts a todo", () => {
    const todo = repo.create("Belajar testing");
    expect(todo).toHaveProperty("id");
    expect(todo).toHaveProperty("title", "Belajar testing");
    expect(todo).toHaveProperty("done", 0);
  });

  it("findAll returns created todos", () => {
    repo.create("Todo A");
    repo.create("Todo B");
    expect(repo.findAll()).toHaveLength(2);
  });
});

```

Auth Test

Simulasi middleware auth:

```

// auth.ts
import { Request, Response, NextFunction } from "express";

export const authMiddleware = (req: Request, res: Response, next: NextFunction) => {
  const token = req.headers.authorization;
  if (!token || token !== "Bearer valid-token") {
    return res.status(401).json({ error: "Unauthorized" });
  }
  next();
};

```

```

};

// auth.test.ts
import { describe, it, expect } from "vitest";
import supertest from "supertest";
import express from "express";
import { authMiddleware } from "../auth";

const app = express();
app.get("/api/protected", authMiddleware, (_req, res) => {
  res.json({ message: "Secret data" });
});

const request = supertest(app);

describe("authMiddleware", () => {
  it("allows valid token", async () => {
    const res = await request
      .get("/api/protected")
      .set("Authorization", "Bearer valid-token");
    expect(res.status).toBe(200);
  });

  it("rejects missing token", async () => {
    const res = await request.get("/api/protected");
    expect(res.status).toBe(401);
  });

  it("rejects wrong token", async () => {
    const res = await request
      .get("/api/protected")
      .set("Authorization", "Bearer wrong-token");
    expect(res.status).toBe(401);
  });
});

```

Test Fixtures

Biar ga nulis data manual tiap test, pake factory function:

```

// fixtures.ts
export const buildTodo = (overrides?: Partial<{ title: string; done: boolean }>) => {
  id: 1,
  title: "Default task",
  done: false,
  ...overrides,
};

```

```

});

export const buildTodoList = (count: number) =>
  Array.from({ length: count }, (_, i) => buildTodo({
    id: i + 1,
    title: `Task ${i + 1}`,
  }));

// fixtures.test.ts
import { describe, it, expect } from "vitest";
import { buildTodo, buildTodoList } from "../fixtures";

describe("buildTodo", () => {
  it("creates todo with defaults", () => {
    const todo = buildTodo();
    expect(todo.title).toBe("Default task");
  });

  it("overrides fields", () => {
    const todo = buildTodo({ title: "Custom", done: true });
    expect(todo.title).toBe("Custom");
    expect(todo.done).toBe(true);
  });
});

describe("buildTodoList", () => {
  it("creates N todos", () => {
    expect(buildTodoList(5)).toHaveLength(5);
  });
});

```

Latihan

1. **User API CRUD test** — Bikin Express app dengan routes: GET /api/users, POST /api/users, GET /api/users/:id, DELETE /api/users/:id. Pake array in-memory. Test semua endpoint termasuk edge case (user not found, duplicate validation, empty body)
2. **Auth-protected todo API** — Gabungin auth middleware + todo CRUD. Test: request tanpa token ditolak (401), request dengan token valid bisa create/read/update/delete. Pastikan test tiap route di-protect
3. **Error handler test** — Bikin global error handler middleware Express. Test: route yang throw error, route dengan invalid JSON body, route dengan parameter invalid (misal id bukan

angka). Pastikan response punya format error konsisten: { error: string, status: number }

4. **Product repository dengan SQLite** — Bikin ProductRepository (create, findAll, findByCategory, updateStock). Pake SQLite in-memory + fixtures. Test: insert product, find by category, update stock jadi 0, hapus product

3. CI/CD dengan GitHub Actions

Apa itu CI/CD?

- **CI (Continuous Integration)**: Setiap push kode, test otomatis dijalankan di server CI
- **CD (Continuous Deployment)**: Setelah test lolos, deploy otomatis ke production

Manfaat:

- Ketahuan langsung kalo ada test failure sebelum merge
- Kode selalu dalam keadaan hijau (test passing)
- Review lebih cepat — ga perlu pull & run test manual

GitHub Actions Workflow Dasar

Bikin file .github/workflows/test.yml:

```
name: Test

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: "npm"
```

- run: npm ci
- run: npm test

Penjelasan tiap bagian:

Bagian	Fungsi
on: push	Trigger tiap push ke branch main
on: pull_request	Trigger tiap PR ke main
runs-on: ubuntu-latest	Runner OS (Linux)
actions/checkout	Clone repo ke runner
actions/setup-node	Install Node.js + npm cache
npm ci	Install dependencies (clean install)
npm test	Jalankan test script

Coverage Reporting

Kirim coverage ke Codecov / Coveralls:

name: Test with Coverage

on: [push, pull_request]

jobs:

test:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4
- uses: actions/setup-node@v4
 - with:
 - node-version: 20
 - cache: "npm"
- run: npm ci
- run: npm test -- --coverage
- name: Upload coverage
 - uses: codecov/codecov-action@v4
 - with:
 - token: \${{ secrets.CODECOV_TOKEN }}
 - files: ./coverage/lcov.info

Di vitest.config.ts:

```
import { defineConfig } from "vitest/config";
```

```
export default defineConfig({
  test: {
    coverage: {
      provider: "v8",
      reporter: ["text", "lcov"],
      include: ["src/**/*.ts"],
      exclude: ["src/**/*.test.ts"],
    },
  },
});
```

Test Matrix (Multi Node Version)

Jalanin test di beberapa versi Node sekaligus:

```
name: Matrix Test

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node-version: [18, 20, 22]

    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${ matrix.node-version }
          cache: "npm"

      - run: npm ci
      - run: npm test
```

Matrix bikin job terpisah buat tiap versi. Kalo salah satu fail, workflow tetap lapor partial failure.

.env untuk CI

Jangan hardcode secret di kode. Pake GitHub Secrets + env variable:

```
jobs:
  test:
    runs-on: ubuntu-latest
```

```

steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-node@v4
    with:
      node-version: 20

  - run: npm ci

  - name: Run tests
    env:
      DATABASE_URL: ${ secrets.DATABASE_URL }
      JWT_SECRET: ${ secrets.JWT_SECRET }
      NODE_ENV: test
    run: npm test

```

Atau bikin file `.env.test` yang di-load otomatis waktu test:

```

# .env.test
DATABASE_URL=sqlite://:memory:
JWT_SECRET=test-secret
NODE_ENV=test

// vitest.config.ts
import { defineConfig } from "vitest/config";

export default defineConfig({
  test: {
    env: {
      DATABASE_URL: "sqlite://:memory:",
      JWT_SECRET: "test-secret",
      NODE_ENV: "test",
    },
  },
});

```

Parallel Test

Vitest otomatis jalanin test file secara paralel. Konfigurasi:

```

// vitest.config.ts
import { defineConfig } from "vitest/config";

export default defineConfig({
  test: {
    pool: "forks", // pake child process (default: threads)
    poolOptions: {

```

```

        forks: {
            singleFork: false,    // multiple forks = paralel
        },
    },
    fileParallelism: true,    // jalanin file paralel
    maxConcurrency: 5,        // max 5 test concurrently
},
});

```

Di GitHub Actions, cache node_modules biar install lebih cepat:

```

- uses: actions/setup-node@v4
  with:
    node-version: 20
    cache: "npm"

- run: npm ci

```

Workflow Lengkap: Test + Lint + Build

```

name: CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: "npm"
      - run: npm ci
      - run: npm run lint

  test:
    needs: lint
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node-version: [18, 20]

```

```

steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-node@v4
    with:
      node-version: ${ matrix.node-version }
      cache: "npm"

  - run: npm ci
  - run: npm test -- --coverage

  - name: Upload coverage
    uses: codecov/codecov-action@v4
    with:
      token: ${ secrets.CODECOV_TOKEN }

build:
  needs: [lint, test]
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: 20
        cache: "npm"
    - run: npm ci
    - run: npm run build

```

Alur: lint dulu → baru test (matrix) → baru build. Kalo lint gagal, test & build skip.

Latihan

1. **Workflow dasar** — Bikin file `.github/workflows/test.yml` yang jalanin test tiap push ke branch main dan tiap PR ke main. Pake Node 20, npm ci, npm test. Push ke GitHub, pastikan workflow muncul
2. **Workflow dengan coverage** — Tambah coverage report pake vitest `run --coverage`. Konfigurasi vitest biar pake v8 provider. Kirim coverage ke Codecov (daftar dulu, dapetin token, simpan di GitHub Secrets)
3. **Workflow matrix** — Ubah workflow supaya test di Node 18, 20, dan 22 sekaligus. Tambah badge di README: `![CI](https://github.com/<user>/<repo>/actions/workflows/test.yml/badge.svg)`

4. **Workflow multi-job** — Bikin workflow dengan 3 job: lint (eslint), test (matrix 18/20 + coverage), build (typescript compile). Pake needs biar job jalan berurutan. Kalo lint gagal, test & build ga jalan